

SD3 — Thread-Safe In-Memory Cache: LRU / LFU (LLD, Oracle machine-coding style)

Oracle: ★★★. Appears as a coding question (Round 1 possible), as an LLD round ("design an in-memory cache with eviction, thread-safe, then scale it"), and as a springboard into "now make it distributed". This is an LLD file: classes, data structures, locks, tests — then the HLD escalation path. Tags: [hot path] on every method — this whole thing is the hot path. ? = follow-up + answer.

Index

0. Time budget (LLD variant)
 1. Why this is hard
 2. Crux
 3. Clarifying questions
 4. FRs
 5. NFRs
 6. BoE
 7. Class design + data structures
 8. API
 9. Data structures detail
 10. Deep dives: LRU $O(1)$ · thread safety · LFU $O(1)$ · TTL + escalation to distributed
 11. Hygiene
 12. Testing + ops
 13. Pop-ups
 14. Trade-offs + defeaters
 15. Wrap-up
 16. Card
 17. References
-

\$0 Time budget (45 min LLD)

Clarify 4 · approach + complexity 4 · single-thread LRU code 12 · thread safety 8 · LFU or TTL (interviewer picks) 8 · tests + escalation 6 → 42

💡 "I'll write the single-threaded $O(1)$ LRU first, prove it with two traces, then add thread safety and discuss what I'd change for LFU and for multi-node."

\$1 Why this is hard

naive: HashMap + timestamp per entry; evict = scan for oldest → $O(n)$ eviction

naive-2: LinkedList of keys, move-to-front on access → $O(n)$ find

- First break: eviction must be $O(1)$ or the cache is slower than the thing it caches at ~10k entries (assume)
- Primitive that fixes it: **hash map + doubly linked list** — map gives $O(1)$ find, list gives $O(1)$ move/remove *given the node*
- Then threads: one lock on everything serializes reads; readers *mutate* recency, so this isn't a read-mostly structure — a RW lock buys nothing
- User wants: "get is $O(1)$, hot keys stay, it never blows memory, and 32 threads don't corrupt it"
- Cost of wrong: corrupted list = infinite loop or NPE under load; too-coarse lock = cache becomes the bottleneck

💡 "This is an **$O(1)$ -under-mutation problem**, because every read is a write to recency."

§2 Crux

- Hard part: maintaining recency order in $O(1)$ while N threads read and write
- Critical points: node identity shared between map and list; every list op is 4 pointer writes done under one lock; capacity enforced on put; no lock held during expensive value load
- Toy vs runnable: toy = LinkedHashMap; runnable = bounded, metrics (hit ratio, evictions), TTL, no stampede on miss
- Metric: hit ratio → backend QPS saved

§3 Clarifying questions

Q	Assumed
Capacity by count or bytes?	count; bytes discussed later
Eviction?	LRU; LFU as follow-up
Thread-safe?	yes, ~32 threads
TTL?	optional follow-up
Value loading on miss inside cache?	yes, <code>computeIfAbsent</code> -style, no stampede
Key/value nullable?	no nulls
Ordering tie-break?	LRU: last accessed; LFU: least frequent, then LRU among ties

§4 FRs

- `get(k)` → value or empty; marks recent
- `put(k,v)` → insert/update; marks recent; evict if over capacity
- `remove(k)`; `size()`; `clear()`
- Extended: TTL per entry; `get(k, loader)`; stats
- Non-goals: persistence; cross-process sharing; weighted sizes in v1

§5 NFRs

Req	Target	Enforced by
get/put	O(1) amortized; < 1 μ s uncontended (order of magnitude)	map + DLL, no allocation on get
Thread safety	linearizable per operation	single lock or striped locks
Memory	$\leq$ capacity entries + O(1) overhead	eviction on put
Contention	no reader starvation	short critical sections

§6 BoE

- Entry overhead: `HashMap.Node` ~32 B + DLL node ~40 B + boxing → ~100 B + key + value; 1M entries × (100 B + 200 B payload) ≈ 300 MB
- Lock cost: uncontended `synchronized` ~20 ns; 32 threads on one lock at 1M ops/s → contention visible; striping needed above ~few hundred k ops/s per JVM
- So what: single lock is correct and fine for < 100k ops/s; say where it stops

§7 Class design + data structures

```

| _____ LruCache<K,V> _____ |
| HashMap<K, Node> map → Node{key, val, prev, next} |
| Node head ⇌ n1 ⇌ n2 ⇌ ... ⇌ tail      (sentinels; head.next = MRU) |
| int capacity ; Lock lock |

```

```
get: map.get → unlink node → insert after head
```

```
put: exists → update + move to front ; else → new node front ; size>cap → unlink tail.prev, map.remove
```

- Components → `Node [hot_path]`: owns key, value, links; identity shared by map and list → `HashMap [hot_path]`: key → node, $O(1)$ → DLL with sentinels `[hot_path]`: recency order; sentinels remove null checks → `Lock [hot_path]`: guards both structures as one invariant

\$8 API

```
public interface Cache<K, V> {  
    Optional<V> get(K key);  
    void put(K key, V value);  
    V getOrLoad(K key, Function<K, V> loader);    // no stampede  
    boolean remove(K key);  
    int size();  
    CacheStats stats();                          // hits, misses, evictions  
}
```

- Contract: null keys/values rejected (`Objects.requireNonNull`); `get` counts as access; `put` on existing key updates value *and* recency

§9 Data structures detail

- Why doubly linked: unlink needs `prev` in $O(1)$; singly linked would need $O(n)$ to find predecessor
- Why sentinels: `head` and `tail` never null → `addFirst` / `unlink` have no branches
- Why not `LinkedHashMap(accessOrder=true)` : it is exactly this (hash table + DLL) — say so, use it in production, hand-roll in interview because they want to see the pointer discipline
- Invariant: `map.size() == list.length`, every map value is in the list, every list node is in the map — write it as a `checkInvariant()` for tests

§10 Deep dives

10.1 LRU $O(1)$, single-threaded (write this first)

```

class Node:
    __slots__ = ("key", "val", "prev", "next")
    def __init__(self, key=None, val=None):
        self.key, self.val = key, val
        self.prev = self.next = None

class LruCache:
    def __init__(self, capacity):
        if capacity <= 0:
            raise ValueError("capacity > 0")
        self.cap = capacity
        self.map = {}
        self.head, self.tail = Node(), Node()      # sentinels; head.next = MRU
        self.head.next, self.tail.prev = self.tail, self.head

    def get(self, key):
        n = self.map.get(key)
        if n is None:
            return None
        self._move_to_front(n)
        return n.val

    def put(self, key, val):
        n = self.map.get(key)
        if n:
            n.val = val
            self._move_to_front(n)
            return
        n = Node(key, val)
        self.map[key] = n
        self._add_first(n)
        if len(self.map) > self.cap:
            lru = self.tail.prev
            self._unlink(lru)
            del self.map[lru.key]

    def _move_to_front(self, n):
        self._unlink(n); self._add_first(n)
    def _add_first(self, n):
        n.next, n.prev = self.head.next, self.head
        self.head.next.prev = n
        self.head.next = n
    def _unlink(self, n):
        n.prev.next, n.next.prev = n.next, n.prev
        n.prev = n.next = None

# Time: O(1) get/put, Space: O(capacity)

```

Production Python shortcut (say it, then hand-roll): `OrderedDict` — `move_to_end(key)` on access, `popitem(last=False)` to evict; or `functools.lru_cache` for functions.

```

import java.util.*;

public class LruCache<K, V> {
    private static final class Node<K, V> {
        K key; V val; Node<K, V> prev, next;
        Node(K k, V v) { key = k; val = v; }
    }

    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final Node<K, V> head = new Node<>(null, null), tail = new Node<>(null, null);

    public LruCache(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity > 0");
        this.capacity = capacity;
        head.next = tail; tail.prev = head;
    }

    public V get(K key) {
        Node<K, V> n = map.get(key);
        if (n == null) return null;
        moveToFront(n);
        return n.val;
    }

    public void put(K key, V val) {
        Node<K, V> n = map.get(key);
        if (n != null) { n.val = val; moveToFront(n); return; }
        n = new Node<>(key, val);
        map.put(key, n);
        addFirst(n);
        if (map.size() > capacity) {
            Node<K, V> lru = tail.prev;
            unlink(lru);
            map.remove(lru.key);
        }
    }

    private void moveToFront(Node<K, V> n) { unlink(n); addFirst(n); }
    private void addFirst(Node<K, V> n) {
        n.next = head.next; n.prev = head;
        head.next.prev = n; head.next = n;
    }
    private void unlink(Node<K, V> n) {
        n.prev.next = n.next; n.next.prev = n.prev;
        n.prev = n.next = null; // helps GC, catches double-unlink in tests
    }
}

```

```
public int size() { return map.size(); }
}
```

Trace to say aloud (cap 2): put(1,A) put(2,B) get(1) put(3,C) → list after each: [1] [2,1] [1,2] [3,1], evicted 2 ✓

Sequence for put (interviewer will ask "walk me through"):

1. Look up node — if present: update value, unlink, addFirst — done; no size change
2. Else create node, map.put, addFirst
3. If size > cap: take tail.prev (the LRU), unlink, map.remove(its key) — if step 3 throws mid-way (it can't here, but say it) map and list disagree → invariant broken → that's why both happen under one lock

? Why store key in the node? → "Eviction needs the key to remove from the map; the list only knows nodes."

? Update on existing key: evict? → "No — size didn't change. Common bug: incrementing before checking existence." ? Capacity 0? → "Reject in constructor; otherwise every put evicts itself and tail.prev == head → unlinking a sentinel corrupts the list."

L5: the evict-then-insert vs insert-then-evict order matters when put of a new key equals capacity: insert-then-evict momentarily holds cap+1 (fine in memory, wrong if capacity is bytes and hard); evict-then-insert must not evict the key being updated. State which you chose and why.

10.2 Thread safety (the Oracle follow-up)

- Option table

Option	Buys	Costs	Choose when
synchronized / ReentrantLock on every method	correct, simple	all ops serialized	< ~100k ops/s, default
ReadWriteLock	—	reads still mutate recency → need write lock → no gain	never for LRU (say this)
Striped locks (N segments, each own map+list)	N× throughput	LRU is per-segment (approximate globally)	> 100k ops/s
ConcurrentHashMap + lock-free list	max throughput	very hard; Caffeine does it with buffered access logs	production → use Caffeine
Copy-on-write	—	O(n) per write	tiny read-only caches

- Pick: ReentrantLock (over synchronized for tryLock / timeouts) wrapping the whole op; then striping if profiling says so

```

from threading import RLock

class ThreadSafeLru(LruCache):
    """Coarse lock: get mutates recency, so a read-write lock buys nothing."""
    def __init__(self, capacity):
        super().__init__(capacity)
        self._lock = RLock()

    def get(self, key):
        with self._lock:
            return super().get(key)

    def put(self, key, val):
        with self._lock:
            super().put(key, val)
# GIL caveat to say: the GIL serializes bytecode, not compound ops –
# get() does 4 pointer writes that must be atomic as a group, so the lock stays.

```

```

public V get(K key) {
    lock.lock();
    try {
        Node<K,V> n = map.get(key);
        if (n == null) { misses.increment(); return null; }
        moveToFront(n); hits.increment();
        return n.val;
    } finally { lock.unlock(); }
}

```

- Stampede-safe loader: **do not hold the lock during load** Sequence:

1. Lock → map lookup → hit? return. Miss → check `inflight: Map<K, CompletableFuture<V>>`; if present, unlock and `join()` it
2. Else create future, put in `inflight`, unlock
3. Run `loader.apply(key)` outside lock — exception → complete future exceptionally, remove from `inflight`, rethrow
4. Lock → `put(key, v)` → remove `inflight` → unlock; complete future → waiters get value

- This is `computeIfAbsent` semantics without blocking the whole cache on a slow loader

Striped version (sketch):

```

final LruCache<K,V>[] segments; // each with own lock
LruCache<K,V> seg(K k){ return segments[(k.hashCode() & 0x7fffffff) % segments.length]; }

```

- Cost: global LRU becomes "LRU within a segment"; a hot segment evicts more. Acceptable; that's how `ConcurrentHashMap` in Java 7 and Guava's cache worked

? *volatile enough?* → "No. `volatile` gives visibility of a single reference; `get` does four pointer writes that must be atomic as a group." ? *Can `get` be lock-free for hits?* → "Not with an exact DLL. Caffeine records reads into a ring buffer and applies them to the list asynchronously — reads are lock-free, recency is eventually consistent. That's the production design; I'd use it, not write it." ? *Deadlock risk?* → "One lock → none. With striping plus an operation touching two segments (e.g., `rename`), lock in segment-index order."

L5: mention memory visibility on the value itself: if `v` is mutable and shared, the cache can't protect it — document values as immutable or defensively copy. And: `ConcurrentHashMap.computeIfAbsent` must not recursively modify the map (throws `IllegalStateException` in Java 9+) — a classic trap if you try to use CHM as the whole cache.

10.3 LFU in O(1) (if asked)

```
freqMap: freq → DLL of nodes (LRU order within freq)
keyMap:  key  → node{key,val,freq}
minFreq: int
get: node.freq++ ; move node from list[f] to list[f+1] ; if list[f] empty && f==minFreq → minFreq++
put new: evict list[minFreq].tail if full ; insert at freq 1 ; minFreq = 1
```

- Two maps + per-frequency DLLs (LeetCode 460 structure). All O(1)
- Why tail of `list[minFreq]`: ties broken by LRU — that's the assumed tie rule
- Defeater: LFU's cold-start problem — a key hit 1,000× yesterday never evicts; production LFU uses decay (Caffeine's TinyLFU with a count-min sketch + window) — name it, don't implement

```

from collections import OrderedDict

class LfuCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.keys = {}                # key -> (val, freq)
        self.freqs = {}              # freq -> OrderedDict(key -> None), LRU within freq
        self.min_freq = 0

    def _touch(self, key):
        val, f = self.keys[key]
        del self.freqs[f][key]
        if not self.freqs[f]:
            del self.freqs[f]
            if self.min_freq == f:
                self.min_freq = f + 1
        self.freqs.setdefault(f + 1, OrderedDict())[key] = None
        self.keys[key] = (val, f + 1)
        return val

    def get(self, key):
        if key not in self.keys:
            return None
        return self._touch(key)

    def put(self, key, val):
        if self.cap <= 0:
            return
        if key in self.keys:
            self.keys[key] = (val, self.keys[key][1])
            self._touch(key)
            return
        if len(self.keys) == self.cap:
            evict, _ = self.freqs[self.min_freq].popitem(last=False)
            if not self.freqs[self.min_freq]:
                del self.freqs[self.min_freq]
            del self.keys[evict]
        self.keys[key] = (val, 1)
        self.freqs.setdefault(1, OrderedDict())[key] = None
        self.min_freq = 1

# Time: O(1) get/put, Space: O(capacity)

```

```

class LfuCache<K,V> {
    private final int cap; private int minFreq = 0;
    private final Map<K, Node<K,V>> keys = new HashMap<>();
    private final Map<Integer, LinkedHashSet<Node<K,V>>> freqs = new HashMap<>();
    // Node has key, val, freq. LinkedHashSet gives O(1) remove + insertion order (LRU within freq).
    // touch(n): remove from freqs[n.freq]; if empty and n.freq==minFreq → minFreq++; n.freq++; add to
    freqs[n.freq]
    // put new: if full → evict first element of freqs[minFreq]; add with freq 1; minFreq = 1
}

```

L5: `LinkedHashSet` instead of hand-rolled DLL is the pragmatic choice under time pressure; say the tradeoff (extra hashing per touch, $\sim 2\times$ memory).

10.4 TTL, size in bytes, and escalation to distributed

- TTL → store `expiresAt` in node; check on `get` (lazy expiry) → return null + remove → active expiry: a `ScheduledExecutor` sampling N random keys per tick (Redis does 20 keys / 100 ms as of Redis 7 docs) — bounded work → defeater: lazy-only TTL means expired entries hold memory until touched or evicted
- Bytes-based capacity: `Weigher<K,V>`; evict while `totalWeight > max`; one huge value can evict everything — cap max entry weight
- Escalation: "make it distributed"
 - Per-node local cache (this) + shared Redis cache-aside — two tiers; local TTL short (1–5 s) to bound staleness
 - Invalidation: on write, `DEL` in Redis + publish `invalidate(key)` on pub/sub; nodes drop local copy — at-most-once delivery → stale until local TTL; that's the accepted bound
 - Consistent hashing across cache nodes if the cache itself must be sharded (→ SD4 KV store)
 - Stampede: singleflight per node + Redis `SET NX` lock with short TTL for cross-node
- 💡 "Local cache is for latency and Redis is for hit ratio; correctness comes from TTL bounds and invalidation, never from assuming the cache is right."

§11 Hygiene (LLD-scaled)

Concern	Here
Null handling	reject; document
Exceptions	loader exception must not poison cache (remove inflight)
Invariant	<code>map.size()==listLen</code> ; assert in tests
Metrics	hits, misses, evictions, load time, size
Memory	bounded by capacity; TTL for staleness; weak keys only if asked
Concurrency	one lock per segment; loader outside lock

§12 Testing + ops

- Unit: LeetCode 146 traces; update-existing doesn't evict; capacity 1; eviction order after mixed get/put

- Invariant fuzz: random ops $\times$ 1M, `checkInvariant()` after each
- Concurrency: 32 threads $\times$ 100k random ops on cap 1,000; assert no exception, `size() <= cap`, invariant holds; run under `-XX:+UseParallelGC` and with `jcstress` if serious
- Stampede: 100 threads `getOrLoad` same key with 200 ms loader $\rightarrow$ loader called once
- Ops (if embedded in a service): expose hit ratio; alarm on hit ratio drop (upstream data shape changed) and on eviction rate spike (capacity too small)

§13 Pop-ups

- *Why not just `LinkedHashMap` with `removeEldestEntry`?* $\rightarrow$ It's correct and I'd ship it wrapped in a lock. Interview wants the pointer logic; production wants Caffeine
- *Why is LRU $O(1)$ but "find LRU" in a naive list $O(n)$?* $\rightarrow$ The map hands you the node; the list never searches
- *Is LRU always the right policy?* $\rightarrow$ No: scans (sequential read of $2\times$ capacity) flush it; ARC / 2Q / TinyLFU resist scans. Say LRU for interview, TinyLFU for production
- *How would you make `get` faster?* $\rightarrow$ Batch recency updates (Caffeine read buffer), or accept approximate LRU (CLOCK: one bit per entry, second-chance) — CLOCK is what Linux page cache and Postgres buffer manager use
- *What breaks with `synchronized` at 1M ops/s?* $\rightarrow$ Lock convoy; threads park/unpark; throughput collapses. Stripe to 16–64 segments
- *Memory leak risk?* $\rightarrow$ Nodes hold keys strongly; if keys are large and cache long-lived, fine (bounded). Leak comes from `inflight` map on loader exception — must remove
- *Make it an LRU with $O(1)$ `getMin`?* $\rightarrow$ Different structure; clarify what "min" means — probably a trick to see if you push back
- *Distributed: how do two nodes stay consistent?* $\rightarrow$ They don't; bound staleness with TTL + invalidation; strong consistency means read-through to the store every time

§14 Trade-offs

Decision	Chosen	Alt	Flips
Structure	HashMap + DLL	LinkedHashMap	production: LinkedHashMap/Caffeine
Locking	one ReentrantLock	striped	$> 100k$ ops/s
Loader	outside lock + inflight map	inside lock	loader is sub- μ s
Policy	LRU	TinyLFU / CLOCK	scan-heavy or huge caches
TTL	lazy + sampled active	lazy only	memory tight

- Defeaters $\rightarrow$ "Use a `ReadWriteLock` for reads" — reads mutate recency; write lock every time $\rightarrow$ "ConcurrentHashMap makes it thread-safe" — the map, not the list; two structures need one lock $\rightarrow$ "Evict on a background thread to make put $O(1)$ " — put already is $O(1)$; background eviction adds a race on capacity

§15 Wrap-up

" $O(1)$ LRU = map to node + doubly linked list with sentinels; the invariant is both structures agree, so both mutate under one lock. Reads mutate recency, so no RW lock; stripe when contended; load outside the lock

with an inflight map to stop stampedes. LFU adds a frequency-bucketed list and minFreq. Production: Caffeine; distributed: local tier + Redis with TTL-bounded staleness and pub/sub invalidation."

§16 Card

- Pitch: `HashMap` → `Node` + sentinel DLL, one lock, loader outside lock, stripe when hot
- Why hard: every read is a recency write
- Numbers: $O(1)$ all ops · ~100 B overhead/entry · ~20 ns uncontended lock · stripe > ~100k ops/s
- Failures: map/list disagree (invariant) · lock convoy · stampede on miss
- Defeaters: RW lock · CHM alone · background eviction
- MVP0 single-thread LRU → MVP1 lock + stats + loader → MVP2 stripes/TTL/LFU
- Done when: write §10.1 from memory in 8 min with the trace; explain why RW lock fails in one sentence

§17 References

- github.com/ben-manes/caffeine — read `BoundedLocalCache`, design wiki (TinyLFU, read buffers)
- `java.util.LinkedHashMap` source — `afterNodeAccess`, `removeEldestEntry`
- LeetCode 146 (LRU), 460 (LFU)
- Redis docs, "How Redis expires keys" (sampled active expiry)
- Caffeine wiki, "Efficiency" (policy comparison, scan resistance)